CHAPTER 1 — ARCHITECTURE OVERVIEW

ARMv8 MMU Fundamentals, Translation Pipeline, and System Context

****

****

1.1 — Purpose of the MMU

****

****

The ARMv8 Memory Management Unit (MMU) provides:

- Virtual memory
- Process isolation
- Memory protection
- Cacheability control
- Device memory mapping
- Virtualization support
- Execution permission enforcement
- Fine-grained control over memory attributes

The MMU is responsible for translating **Virtual Addresses (VA)** generated by the CPU into **Physical Addresses (PA)** used by memory and devices.

In virtualization environments, translation is split into two stages:

- **Stage-1:** VA → IPA (Intermediate Physical Address)
- **Stage-2:** IPA → PA (Physical Address)

This two-stage model enables hypervisors to control guest memory access.

****

****

1.2 — The Two-Stage Translation Model

****

****

ARMv8 uses a layered translation pipeline:

```
Virtual Address (VA)
        │
        ▼
Stage-1 Translation (EL0/EL1)
        │
        ▼
Intermediate Physical Address (IPA)
        │
        ▼
Stage-2 Translation (EL2)
        │
        ▼
Physical Address (PA)
```

Stage-1 (Guest OS)

- Controlled by EL1 (kernel)
- Produces IPA
- Defines guest permissions
- Defines guest memory attributes

Stage-2 (Hypervisor)

- Controlled by EL2
- Produces final PA
- Enforces VM isolation
- Overrides guest permissions
- Overrides guest memory attributes

Teaching Note: Stage-2 is always authoritative. Even if Stage-1 allows access, Stage-2 can deny it.

****

****

1.3 — Why Two Stages Exist

****

****

Two-stage translation enables:

Virtualization

Each VM sees its own “physical” memory (IPA), but the hypervisor maps IPA → real PA.

Security

Hypervisor can:

- deny access
- force non-exec
- force read-only
- force Device memory type

- isolate VMs

Flexibility

Guest OS can manage its own virtual memory without knowing real hardware layout.

****

****

1.4 — MMU Responsibilities

****

****

The MMU performs:

- Address translation
- Permission checking
- Memory attribute selection
- Shareability enforcement
- TLB caching
- Walk caching
- Fault generation
- Synchronization with barriers

Implementation Note: All of these operations must be deterministic and safe. Incorrect page table updates can corrupt TLB state and cause undefined behavior.

****

****

1.5 — Privilege Levels and Translation Context

****

****

ARMv8 privilege levels:

- **EL0:** User mode
- **EL1:** Kernel / OS
- **EL2:** Hypervisor
- **EL3:** Secure Monitor (TrustZone)

Translation Contexts

Level	Controls	Translation
EL0	User code	Uses EL1 Stage-1 tables
EL1	Kernel	Defines Stage-1 tables
EL2	Hypervisor	Defines Stage-2 tables
EL3	Secure monitor	Optional secure translation

Teaching Note: EL0 and EL1 always go through Stage-1. EL2 may bypass Stage-1 depending on configuration.

****

****

1.6 — MMU Interaction with Caches

****

****

Memory attributes (MAIR) determine:

- cacheability
- write-back vs write-through
- device ordering
- shareability

The MMU ensures that memory accesses obey the rules defined by:

- MAIR_EL1
- MAIR_EL2
- SH bits
- TCR defaults

Implementation Note: Incorrect memory attributes can break device drivers, DMA, and SMP coherence.

****

****

1.7 — MMU Interaction with TLB

****

****

TLB stores:

- VA → PA mappings
- permissions
- attributes
- ASID
- VMID

TLB hits avoid page table walks.

TLB misses trigger:

- Stage-1 walk
- Stage-2 walk

Teaching Note: TLB entries are tagged with ASID (process) and VMID (VM). This allows multiple processes and VMs to coexist without flushing the TLB.

****

****

1.8 — MMU Interaction with Barriers

****

****

Barriers ensure:

- page table writes are visible
- TLB invalidations complete
- instruction pipeline sees new mappings

Required barriers:

- **DSB** — complete memory operations
- **DMB** — order memory operations
- **ISB** — flush instruction pipeline

Implementation Note: Skipping barriers causes silent memory corruption.

****

****

1.9 — MMU Interaction with Page Table Updates

****

****

Page table updates must follow:

1. Write descriptor
2. DSB
3. TLBI
4. DSB
5. ISB

This ensures:

- no stale translations
- no mixed attributes
- no mixed permissions
- no walk cache corruption

****

****

1.10 — Break-Before-Make (BBM)

****

****

BBM rule:

You must invalidate a descriptor before replacing it with another valid descriptor.

Required when changing:

- block → page
- page → block
- block → table
- table → block
- permissions
- memory attributes
- physical address

Teaching Note: BBM prevents TLB corruption and undefined behavior.

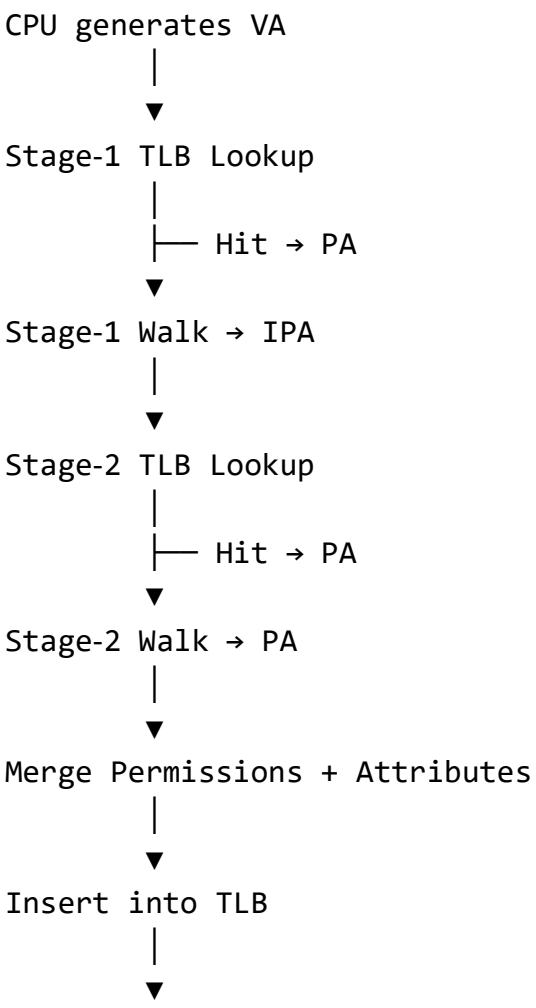
****

****

1.11 — Full Translation Pipeline (High-Level Diagram)

****

****



CPU Access

****

****

1.12 — Chapter Summary

****

****

This chapter introduced:

- MMU purpose
- two-stage translation
- privilege levels
- TLB interaction
- barrier requirements
- BBM safety rule
- high-level pipeline